

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool

Weightage:40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

CaseStudy

Code of Conduct:

I D.HUSSAINIAH(1924253000certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:D.HUSSAINIAH

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail. Analyze the problem and explain how preprocessing enhances edge detection results.

1. Understanding of the Case

Edge detection algorithms such as Sobel, Prewitt, Canny, and Laplacian of Gaussian identify edges by locating pixel locations where intensity changes sharply — mathematically, where the image gradient magnitude $|\nabla I|$ is large. In a low-contrast image, neighbouring regions (object and background, or two adjacent objects) have very similar grey-level or colour intensities. The intensity difference across a true edge is small, so the gradient magnitude at genuine boundaries is not much larger than the gradient magnitude produced by sensor noise or minor illumination variation.

This creates two linked failures. First, weak-but-real edges fall below the detector's gradient threshold and are missed (false negatives). Second, if the threshold is lowered to catch these weak edges, noise-induced gradient fluctuations elsewhere in the image are also picked up as spurious edges (false positives). The core issue, therefore, is not a flaw in the edge-detection mathematics itself, but a signal-to-background-contrast problem in the input image that the detector cannot distinguish from noise.

2. Application of Concepts

Several image-processing concepts explain and address this scenario:

- Gradient-based detection: Sobel/Prewitt approximate $\partial I/\partial x$ and $\partial I/\partial y$ using convolution kernels; low contrast directly shrinks these derivative values.
- Contrast/Histogram theory: contrast is related to the spread of the intensity histogram. A low-contrast image has pixel values clustered in a narrow band instead of spanning the full dynamic range (0–255 for 8-bit images).
- Canny's multi-stage pipeline: Gaussian smoothing, gradient computation, non-maximum suppression, and hysteresis thresholding — the hysteresis (high/low threshold) stage is especially sensitive to weak-edge/noise ambiguity in low-contrast inputs.
- Preprocessing as a contrast-enhancement stage that operates on the image before gradient computation, so the detector receives a signal with a wider, better-separated intensity distribution.

3. Analysis

The failure can be analyzed as a competition between two signals of comparable magnitude: the true-edge gradient (small, because contrast is low) and the noise gradient (roughly constant, determined by sensor/ISO noise). Edge detection effectively performs threshold-based separation of these two; when their magnitudes overlap, no single global threshold can cleanly separate them. Reducing the threshold recovers real edges but also large numbers of noise edges; raising it removes noise edges but also removes real ones. This is a precision–recall trade-off intrinsic to the raw image, not to the algorithm.

This also explains why simply 'tuning parameters' of the edge detector (kernel size, threshold values) gives only limited improvement — the underlying image itself lacks sufficient separability between edge and non-edge pixels. The correct intervention point is therefore upstream of the detector, in preprocessing, where the intensity distribution itself can be reshaped.

4. Solution Design — Preprocessing Techniques

Histogram Equalization: redistributes pixel intensities so the histogram approaches a uniform distribution, stretching the narrow low-contrast band across the full dynamic range and amplifying weak intensity differences at true edges.

Contrast Limited Adaptive Histogram Equalization (CLAHE): applies histogram equalization locally, in small tiles, with a clip limit to prevent noise over-amplification — more effective than global equalization for images with non-uniform lighting.

Gaussian Smoothing before gradient computation: (as in the Canny pipeline) suppresses high-frequency noise so it doesn't dominate the now-amplified gradient signal, without significantly blurring genuine low-frequency edges.

Gamma Correction / Intensity Stretching: non-linearly remaps pixel values to expand contrast in the intensity range where most scene content lies.

Unsharp Masking: subtracts a blurred version of the image from the original to boost high-frequency (edge-region) content before detection.

Adaptive thresholding (e.g., Otsu's method): chosen based on the enhanced image's statistics rather than a fixed global value, further improving separation between edge and non-edge gradients.

Recommended pipeline: Gaussian denoise → CLAHE contrast enhancement → Canny edge detection with hysteresis thresholding. This order denoises first (so equalization doesn't amplify noise), enhances

contrast second, and detects edges last on a signal where true boundaries now produce gradients well above the noise floor.

5. Clarity / Conclusion

Low contrast causes edge detectors to fail because true-edge gradients become comparable in magnitude to noise gradients, making any single threshold ineffective. Preprocessing (denoising + contrast enhancement via CLAHE/histogram equalization) reshapes the intensity distribution so genuine edges produce distinctly larger gradients than noise, restoring reliable detection without needing to alter the detector's core algorithm.

2. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints. Analyze the issue and suggest methods to improve feature selection.

1. Understanding of the Case

Corner detectors such as Harris, Shi-Tomasi (Good Features to Track), and FAST identify points where the local image gradient changes significantly in two orthogonal directions — i.e., points of high 'cornerness', measured via the eigenvalues of the second-moment (structure) matrix, or, for FAST, a ring of pixels that differ sufficiently from the centre pixel. Highly textured regions (foliage, fabric patterns, gravel, brick walls) contain dense, repetitive micro-variations in intensity. Nearly every local neighbourhood in such a region satisfies the corner criterion, so the detector reports an extremely large number of keypoints, many of which are redundant, unstable, or not semantically meaningful.

The core issue is that texture produces a high density of locally valid but globally uninformative corners: they are mathematically real corners but do not correspond to distinct, well-separated structural features useful for tasks like matching, tracking, or 3D reconstruction.

2. Application of Concepts

- Harris/Shi-Tomasi corner response: based on eigenvalues (λ_1, λ_2) of the structure tensor; both large $\Rightarrow$ corner. Texture produces many regions where both eigenvalues are large simultaneously.
- Non-maximum suppression (NMS): a spatial filtering concept that keeps only the locally strongest response within a neighbourhood, removing closely clustered duplicate detections.
- Scale-space theory: features detected at only one scale over-represent fine texture; multi-scale analysis distinguishes structural corners (stable across scales) from texture noise (present mainly at fine scales).
- Feature response ranking / thresholding: using the corner response score (Harris R-value, FAST score) to keep only the strongest, most distinctive keypoints.

3. Analysis

Excessive keypoints degrade the system in several concrete ways: (a) computational cost rises sharply for downstream steps such as descriptor computation and matching; (b) matching accuracy falls, because many texture-corners look locally similar to each other, increasing the ambiguity/mismatch rate; (c)

storage and real-time performance (e.g., in SLAM or tracking) suffer due to redundant data. The root cause is that the detector has no built-in notion of 'global distinctiveness' — it only evaluates local gradient structure, so uniform response thresholds cannot separate meaningful structural corners from repetitive texture responses that happen to cross the same threshold.

A useful analytical framing: texture-region keypoints tend to cluster spatially (many corners in a small area) and have similar response magnitudes, whereas structurally significant corners (object boundaries, junctions) are typically sparser and often stronger/more isolated. This spatial and magnitude distribution difference is exploited by the solutions below.

4. Solution Design — Improving Feature Selection

Adaptive Non-Maximum Suppression (ANMS): retains a keypoint only if it is the strongest within a radius, and enforces spatial spreading across the image so keypoints are not clustered purely where texture is densest.

Response thresholding with higher/adaptive cut-off: raise the Harris/Shi-Tomasi corner-response threshold, or set it adaptively (e.g., relative to the image's own response statistics) to keep only strong, distinctive corners.

Limiting the number of keypoints (Top-N selection): rank all detected corners by response strength and retain only the top N, as done in Shi-Tomasi's 'good features to track'.

Grid-based / spatial-binning selection: divide the image into a grid and keep only the best few keypoints per cell, ensuring a uniform, non-redundant spatial distribution instead of texture-dominated clustering.

Scale-space / multi-scale filtering: discard keypoints that don't persist across multiple scales, since texture-induced corners are often scale-unstable while true structural corners are scale-consistent.

Descriptor-distinctiveness filtering: after computing descriptors (e.g., SIFT/ORB), remove keypoints whose descriptors are near-duplicates of neighbouring keypoints, since these add no discriminative value for matching.

Recommended pipeline: Shi-Tomasi/Harris detection → response-based thresholding → grid-based Top-N selection with ANMS → optional multi-scale consistency check. This keeps a sparse, well-distributed, high-quality keypoint set even in dense texture.

5. Clarity / Conclusion

Highly textured regions generate excessive corners because texture satisfies the local corner criterion almost everywhere. The fix is not to change the detector's fundamental mathematics but to add distinctiveness- and distribution-aware selection on top of it — via adaptive non-maximum suppression, response thresholding, Top-N/grid-based selection, and multi-scale consistency checks — yielding a sparse, informative, and matching-friendly keypoint set.

3. Feature Matching with Noise

Feature matching between two images fails due to noise in one image. Analyze the impact of noise and explain how preprocessing improves matching accuracy.

1. Understanding of the Case

Feature matching (e.g., using SIFT, SURF, ORB descriptors with nearest-neighbour or ratio-test matching) relies on computing a descriptor vector for each keypoint that should be nearly identical for the same physical point seen in two images, and clearly different for unrelated points. When one image contains significant noise (sensor noise, compression artifacts, low-light grain), the pixel intensities around each keypoint are perturbed, which corrupts both keypoint localization and the descriptor values computed from local gradients/intensity patterns. As a result, corresponding points in the two images no longer produce sufficiently similar descriptors, and the matcher either fails to find a match, finds an incorrect match, or the keypoint itself shifts/disappears due to noise-corrupted local structure.

2. Application of Concepts

- Descriptor invariance assumptions: SIFT/SURF/ORB descriptors assume local gradient/intensity patterns are stable; noise violates this local stability assumption.
- Keypoint detection stability: noise alters local extrema in scale-space (SIFT) or corner responses (Harris/FAST), causing keypoints to be detected at slightly different, or entirely different, locations between the two images.
- Distance metrics in matching: Euclidean/Hamming distance between descriptors increases due to noise-induced descriptor drift, pushing true matches past the acceptance threshold or ratio test (Lowe's ratio test).
- Signal-to-noise ratio (SNR): matching accuracy is fundamentally governed by how much of the descriptor's variation is genuine structure versus noise.

3. Analysis

Noise affects the matching pipeline at three separate stages, compounding the failure: (1) Detection stage — noise can create spurious local extrema (false keypoints) or shift true keypoints' sub-pixel location; (2) Description stage — gradient-based descriptors computed over a noisy neighbourhood have altered magnitude/orientation histograms, so the same physical point yields different descriptor vectors in the clean vs. noisy image; (3) Matching stage — increased descriptor distance for true

correspondences reduces the gap between correct and incorrect matches, raising both the false-negative rate (correct matches rejected) and false-positive rate (incorrect matches accepted), and undermining the reliability of the ratio test.

Because noise is typically image-specific and asymmetric here (only one image is noisy), the two images' keypoint sets and descriptor statistics become inconsistent — this is worse than if both images had similar noise, since symmetric noise at least preserves relative similarity, whereas asymmetric noise breaks the invariance the matcher depends on.

4. Solution Design — Preprocessing to Improve Matching

Gaussian / Bilateral filtering: denoise the affected image before feature detection; bilateral filtering is preferred as it smooths noise while preserving edges, which are critical for keypoint/descriptor quality.

Median filtering: effective for impulse/salt-and-pepper noise, removing outlier pixel values without excessive blurring.

Non-local means denoising: exploits redundant patch similarity across the image for stronger noise suppression while retaining fine texture needed for descriptors.

Wavelet-domain denoising: separates noise (high-frequency, low-correlation) from structure in the wavelet domain, thresholding out noise coefficients.

Consistent preprocessing on both images: apply the same smoothing/normalization pipeline to both images (not just the noisy one) so keypoint scale-space behaviour and descriptor statistics remain comparable.

Robust matching strategies (complementary): Lowe's ratio test, cross-check (mutual nearest neighbour) matching, and RANSAC-based geometric verification to reject outlier matches that survive despite noise.

Recommended pipeline: apply bilateral or non-local-means denoising to the noisy image (matched by light smoothing on the clean image for consistency) → re-run keypoint detection and descriptor extraction → match with ratio test + mutual cross-check → RANSAC to filter remaining outliers.

5. Clarity / Conclusion

Noise degrades feature matching by corrupting keypoint localization and descriptor values at the source, which increases descriptor distance for genuine correspondences and blurs the distinction between correct and incorrect matches. Denoising the image before detection — using edge-preserving and RANSAC-based outlier rejection.

4. PCA and Information Loss

A system applies PCA aggressively and loses important feature details. Analyze the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

1. Understanding of the Case

Principal Component Analysis (PCA) reduces the dimensionality of feature vectors (e.g., image patches, descriptor sets, or eigenfaces) by projecting data onto the top-k eigenvectors of the data's covariance matrix — the directions (principal components) of maximum variance. 'Applying PCA aggressively' means choosing a very small number of components k , far below what is needed to preserve the data's meaningful variance. While this yields a highly compact representation and fast computation, it discards the eigenvectors associated with smaller (but not negligible) eigenvalues, which often still encode genuine, task-relevant structure — fine details, subtle texture, or discriminative variations between classes. The system therefore becomes fast but under-informative, leading to poor downstream performance (e.g., degraded recognition/classification/reconstruction accuracy).

2. Application of Concepts

- Eigenvalue spectrum: each principal component's eigenvalue quantifies the variance it captures; total variance = sum of all eigenvalues.
- Explained variance ratio: fraction of total variance retained by the first k components = $(\sum \text{of top-}k \text{ eigenvalues}) / (\sum \text{of all eigenvalues})$.
- Reconstruction error: the information discarded by truncating to k components is proportional to the sum of the discarded eigenvalues; reconstructing from k components introduces error growing as k decreases.
- Curse of dimensionality vs. under-fitting: too many components retain noise and redundant dimensions (harder to generalize); too few components remove discriminative signal (under-fitting) — this is the core dimensionality-reduction trade-off.

3. Analysis

The trade-off is fundamentally a bias–variance style balance applied to representation size. Retaining very few components (aggressive PCA) minimizes computational cost and can suppress noise (since noise is often spread across low-variance components), but risks discarding components that, while individually lower-variance, are collectively important for distinguishing between classes or reconstructing fine detail — this is especially damaging in tasks like face recognition or fine-grained image matching, where subtle differences carry the discriminative signal. Conversely, retaining too many components keeps noise and redundant, highly correlated dimensions, increasing computational cost and the risk of overfitting in downstream classifiers, without proportionally improving accuracy.

The relationship between k and retained information is typically not linear: eigenvalues from real image data usually decay rapidly at first (the first few components capture most global variance — overall illumination, coarse shape) and then flatten into a long tail of smaller eigenvalues that still cumulatively encode fine detail. Aggressive truncation cuts into this tail too early, right where fine but meaningful detail lives, which explains why the system in this scenario 'loses important feature details' even though it may still look reasonably efficient on paper.

4. Solution Design — Justifying Optimal Component Selection

Explained-variance threshold: choose the smallest k such that cumulative explained variance $\geq$ a target (commonly 90–99%, depending on task sensitivity), read directly off the eigenvalue spectrum.

Scree plot / elbow method: plot eigenvalues (or explained variance) against component index and select k at the 'elbow' where marginal variance gain per additional component drops sharply — balancing compactness against diminishing returns.

Cross-validated task performance: for a specific downstream task (classification/matching), sweep k and choose the value that maximizes validation accuracy/F1 rather than relying on variance alone, since the 'informative' components for a task are not always the highest-variance ones.

Reconstruction-error budget: set a maximum acceptable reconstruction error (e.g., mean squared error between original and reconstructed feature) and pick the minimum k that satisfies it.

Kaiser's criterion / average-eigenvalue rule: retain components whose eigenvalue exceeds the average eigenvalue, as a quick heuristic for 'components carrying more than their fair share of variance'.

Domain-aware validation: for visually/perceptually sensitive tasks, complement quantitative variance metrics with qualitative inspection of reconstructed samples at candidate k values to confirm fine detail is preserved.

Recommended approach: compute the full eigenvalue spectrum, use the 90–95% cumulative-explained-variance rule (or scree-plot elbow) to shortlist a candidate range of k , then fine-tune k using cross-validated downstream task accuracy — rather than picking k arbitrarily small purely for speed/storage gains.

5. Clarity / Conclusion

Aggressive PCA loses information because it discards low-variance-but-informative components, particularly the fine-detail tail of the eigenvalue spectrum. The correct number of components is not a fixed constant but a trade-off point balanced against task requirements — determined systematically via cumulative explained variance, scree-plot elbow analysis, and cross-validated downstream performance, rather than chosen for compactness alone.

5. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection. Analyze the issue and explain how appropriate feature descriptors improve matching performance.

1. Understanding of the Case

Feature-based vision pipelines have two logically separate stages: detection (locating distinctive keypoints — corners, blobs, extrema) and description (encoding the local appearance around each keypoint into a vector for comparison). This scenario states that detection is working correctly — keypoints are being located accurately and repeatably — but matching still fails. This isolates the fault to the description stage: the chosen descriptor either does not capture enough discriminative or invariant information about each keypoint's neighbourhood, or is mismatched to the conditions of the images being compared (e.g., a rotation-sensitive descriptor used on rotated images, or a low-dimensional binary descriptor used where fine discrimination is required).

2. Application of Concepts

- Descriptor invariance properties: descriptors differ in robustness to scale (SIFT, SURF), rotation (SIFT's dominant-orientation assignment, ORB's oriented BRIEF), illumination (gradient-normalized descriptors), and affine/viewpoint change (ASIFT-style extensions).
- Descriptor dimensionality and representation: SIFT (128-D float, gradient histograms) vs. SURF (64/128-D, Haar-wavelet based) vs. ORB/BRIEF (binary strings, Hamming-distance matching) — each trades discriminative power against speed and storage differently.
- Distance metric compatibility: floating-point descriptors (SIFT/SURF) are compared with Euclidean distance; binary descriptors (ORB/BRIEF/BRISK) require Hamming distance — using the wrong metric for a descriptor type degrades matching regardless of descriptor quality.
- Discriminative power vs. generality: a descriptor must be distinctive enough to separate different keypoints, yet invariant enough that the same physical point yields a similar descriptor under viewpoint/illumination change — this is the essential design tension in descriptor selection.

3. Analysis

Because detection is confirmed correct, the analysis should trace exactly where descriptor mismatch causes failure: (a) Insufficient invariance — if the two images differ in rotation, scale, or illumination and a descriptor lacking that invariance (e.g., a plain intensity-patch descriptor) is used, the same keypoint

produces very different vectors in each image, so true matches have large descriptor distance; (b) Insufficient distinctiveness — a descriptor that is too generic (short binary string in a highly repetitive scene) cannot separate visually similar keypoints, causing false matches; (c) Metric mismatch — applying Euclidean distance to a binary descriptor's bit-string (or vice versa) produces meaningless distances, causing the matcher to rank matches essentially randomly.

This demonstrates that detection accuracy alone does not guarantee matching success — the descriptor is the actual carrier of the 'identity' of a keypoint across images, and if it fails to represent that identity robustly and distinctively, correct detection is wasted downstream.

4. Solution Design — Selecting Appropriate Descriptors

Match descriptor invariance to expected image variation: use SIFT or SURF when scale/rotation/illumination changes are expected (e.g., different viewpoints, zoom levels); use simpler descriptors only when conditions are controlled and near-identical.

Use ORB/BRISK/FREAK for real-time or resource-constrained systems: these binary descriptors are fast and rotation-invariant (via orientation assignment), provided Hamming-distance matching is used consistently.

Ensure metric consistency: Euclidean/L2 distance (often with a k-d tree or FLANN for speed) for float descriptors; Hamming distance (often with an LSH index) for binary descriptors.

Apply Lowe's ratio test and mutual cross-check: regardless of descriptor type, to reject ambiguous matches where the descriptor's discriminative power is locally weak.

Geometric verification (RANSAC): with a homography/fundamental-matrix model, as a final filter, since even a well-chosen descriptor will produce some outlier matches that only geometric consistency can catch.

Empirical benchmarking: evaluate candidate descriptors on a validation set representative of the deployment conditions (illumination range, expected rotation/scale) and select based on measured precision/recall of matches, not descriptor popularity alone.

Recommended approach: characterize the expected image-to-image variation (rotation, scale, illumination, viewpoint) → select a descriptor whose invariance properties match that variation (e.g., SIFT for general-purpose robustness, ORB for real-time constraints) → use the metric appropriate to that descriptor type → apply ratio test, cross-check, and RANSAC to finalize robust matches.

5. Clarity / Conclusion

When detection is accurate but matching fails, the fault lies in the description stage: an incorrectly chosen descriptor either lacks the invariance needed for the imaging conditions, lacks sufficient distinctiveness, or is paired with an incompatible distance metric. Selecting a descriptor whose invariance and dimensionality match the task — and pairing it with the correct distance metric and outlier-rejection steps (ratio test, cross-check, RANSAC) — restores reliable matching performance without needing to alter the (already correct) detection stage.